CSA09 – Programming in Java

CO5 Individual Assignment — Application-Based Problem Solving

1. HEADER

Name	K. SHALINI
Roll No.	17
Course Code & CO	CSA09, CO5 – JDBC and Database-Driven Applications
Assignment Set	Application-Based Problem Solving – Question 17
GitHub Repository	[https://github.com/Shalini387/CSA0926-JAVA]
Submission Date	[29-08-2026]

Full Text of Assigned Question:

Q17. Job Portal / Recruitment Management System — An HR recruitment team needs to track job openings and candidate applications against them, along with each application's current status. Build a Java application with a "jobs" table (job_id, title, department) and an "applications" table (app_id, job_id, candidate_name, status) that lets candidates apply to a job, lets HR update an application's status (Applied/Shortlisted/Rejected/Hired), and searches applications by job or status.

Core Modules: Post Job Opening, Apply for Job, Update Application Status, Search Applications by Job/Status, Shortlisted Candidates Report.

JDBC Requirement: PreparedStatement throughout; demonstrate a join query between jobs and applications to show which candidates applied to which job title (not just job_id).

What Students Must Present: An application submission and status-update demo, a filtered search by status, and the joined shortlisted-candidates report.

2. PROBLEM UNDERSTANDING

The program must let an HR team post job openings, let candidates apply against them, let HR update each application's status through its lifecycle, and let HR search or report on applications — displaying the job's title rather than its internal ID. It tests understanding of a one-to-many relationship between two tables (one job can have many applications), how to enforce that relationship with a foreign key, and how to reconstruct human-readable results with a JOIN instead of exposing raw foreign key values.

3. APPROACH / DESIGN NOTES

- Used two tables: jobs(job_id, title, department) and applications(app_id, job_id, candidate_name, status), with job_id in applications as a foreign key referencing jobs.
- Used PreparedStatement for every INSERT/UPDATE/SELECT — no string concatenation, so the queries are safe from SQL injection regardless of what a user types into the candidate name or job title fields.
- Before inserting an application, the code checks that the given job_id actually exists in jobs, so a candidate can't apply to a non-existent job — caught in application logic in addition to the database's own foreign key constraint.
- The Search and Shortlisted Report modules both use a JOIN between applications and jobs so results show the job's title, not just its numeric job_id, as the assignment specifically requires.
- Alternative considered: storing job_title directly inside applications to avoid the join. Rejected — this duplicates data and would go stale if a job's title were ever edited; normalizing into two tables and joining is the correct relational design.
- No transaction (commit/rollback) or CallableStatement is required for Q17 — it isn't in the transaction list (5, 8, 15, 22, 25, 35, 38, 40) or the stored-procedure list (2, 29, 32), so plain PreparedStatement CRUD is what's expected here.

4. SOURCE CODE

SQL table-creation script (run first, in MySQL):

```
CREATE DATABASE IF NOT EXISTS job_portal;
USE job_portal;

CREATE TABLE jobs (
    job_id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(100) NOT NULL,
    department VARCHAR(100) NOT NULL
);

CREATE TABLE applications (
    app_id INT PRIMARY KEY AUTO_INCREMENT,
    job_id INT NOT NULL,
    candidate_name VARCHAR(100) NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Applied',
    FOREIGN KEY (job_id) REFERENCES jobs(job_id)
);
```

Java source (JobPortalApp.java — Swing GUI + JDBC):

```
import javax.swing.*;
import java.awt.*;
import java.sql.*;

public class JobPortalApp extends JFrame {

    private static final String URL  = "jdbc:mysql://localhost:3306/job_portal";
    private static final String USER = "root";
    private static final String PASS = "yourpassword"; // change to your MySQL password
```

```
    private JTextField jobTitleField, jobDeptField;
    private JTextField appJobIdField, appNameField;
    private JTextField updateAppIdField;
    private JComboBox<String> statusDropdown;
    private JTextField searchJobIdField;
    private JComboBox<String> searchStatusDropdown;
    private JTextArea outputArea;

    public JobPortalApp() {
        setTitle("Job Portal / Recruitment Management System");
        setSize(700, 650);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        JTabbedPane tabs = new JTabbedPane();
        tabs.add("Post Job", buildPostJobPanel());
        tabs.add("Apply for Job", buildApplyPanel());
        tabs.add("Update Status", buildUpdateStatusPanel());
        tabs.add("Search Applications", buildSearchPanel());
        tabs.add("Shortlisted Report", buildReportPanel());

        outputArea = new JTextArea(10, 60);
        outputArea.setEditable(false);
        JScrollPane scroll = new JScrollPane(outputArea);

        add(tabs, BorderLayout.CENTER);
        add(scroll, BorderLayout.SOUTH);
    }

    private Connection connect() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASS);
    }
```

```
    // ---------- Core Module 1: Post Job Opening ----------
    private JPanel buildPostJobPanel() {
        JPanel panel = new JPanel(new GridLayout(3, 2, 5, 5));
        jobTitleField = new JTextField();
        jobDeptField = new JTextField();
        JButton postBtn = new JButton("Post Job");

        panel.add(new JLabel("Job Title:"));
        panel.add(jobTitleField);
        panel.add(new JLabel("Department:"));
        panel.add(jobDeptField);
        panel.add(new JLabel(""));
        panel.add(postBtn);

        postBtn.addActionListener(e -> postJob());
        return panel;
    }

    private void postJob() {
        String title = jobTitleField.getText().trim();
        String dept = jobDeptField.getText().trim();
        if (title.isEmpty() || dept.isEmpty()) {
            outputArea.setText("Error: Job title and department are required.");
            return;
        }
        String sql = "INSERT INTO jobs (title, department) VALUES (?, ?)";
        try (Connection con = connect();
             PreparedStatement ps = con.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, title);
            ps.setString(2, dept);
```

```
            ps.executeUpdate();
            try (ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) {
                    outputArea.setText("Job posted successfully. job_id = " + rs.getInt(1));
                }
            }
            jobTitleField.setText("");
            jobDeptField.setText("");
        } catch (SQLException ex) {
            outputArea.setText("Error posting job: " + ex.getMessage());
        }
    }

    // ---------- Core Module 2: Apply for Job ----------
    private JPanel buildApplyPanel() {
        JPanel panel = new JPanel(new GridLayout(3, 2, 5, 5));
        appJobIdField = new JTextField();
        appNameField = new JTextField();
        JButton applyBtn = new JButton("Apply");

        panel.add(new JLabel("Job ID:"));
        panel.add(appJobIdField);
        panel.add(new JLabel("Candidate Name:"));
        panel.add(appNameField);
        panel.add(new JLabel(""));
        panel.add(applyBtn);

        applyBtn.addActionListener(e -> applyForJob());
        return panel;
    }

    private void applyForJob() {
```

```
        String jobIdText = appJobIdField.getText().trim();
        String name = appNameField.getText().trim();
        if (jobIdText.isEmpty() || name.isEmpty()) {
            outputArea.setText("Error: Job ID and candidate name are required.");
            return;
        }
        int jobId;
        try {
            jobId = Integer.parseInt(jobIdText);
        } catch (NumberFormatException nfe) {
            outputArea.setText("Error: Job ID must be a number.");
            return;
        }

        // Verify job exists before inserting (invalid/failure test case)
        String checkSql = "SELECT job_id FROM jobs WHERE job_id = ?";
        String insertSql = "INSERT INTO applications (job_id, candidate_name, status)
VALUES (?, ?, 'Applied')";
        try (Connection con = connect()) {
            try (PreparedStatement check = con.prepareStatement(checkSql)) {
                check.setInt(1, jobId);
                try (ResultSet rs = check.executeQuery()) {
                    if (!rs.next()) {
                        outputArea.setText("Error: No job exists with job_id " + jobId);
                        return;
                    }
                }
            }
            try (PreparedStatement ps = con.prepareStatement(insertSql,
Statement.RETURN_GENERATED_KEYS)) {
                ps.setInt(1, jobId);
                ps.setString(2, name);
```

```
            ps.executeUpdate();
            try (ResultSet rs = ps.getGeneratedKeys()) {
                if (rs.next()) {
                    outputArea.setText("Application submitted. app_id = " + rs.getInt(1) + ",
status = Applied");
                }
            }
        }
        appJobIdField.setText("");
        appNameField.setText("");
    } catch (SQLException ex) {
        outputArea.setText("Error applying: " + ex.getMessage());
    }
  }

  // ---------- Core Module 3: Update Application Status ----------
  private JPanel buildUpdateStatusPanel() {
    JPanel panel = new JPanel(new GridLayout(3, 2, 5, 5));
    updateAppIdField = new JTextField();
    statusDropdown = new JComboBox<>(new String[]{"Applied", "Shortlisted",
"Rejected", "Hired"});
    JButton updateBtn = new JButton("Update Status");

    panel.add(new JLabel("Application ID:"));
    panel.add(updateAppIdField);
    panel.add(new JLabel("New Status:"));
    panel.add(statusDropdown);
    panel.add(new JLabel(""));
    panel.add(updateBtn);

    updateBtn.addActionListener(e -> updateStatus());
    return panel;
```

```
    }

    private void updateStatus() {
        String appIdText = updateAppIdField.getText().trim();
        if (appIdText.isEmpty()) {
            outputArea.setText("Error: Application ID is required.");
            return;
        }
        int appId;
        try {
            appId = Integer.parseInt(appIdText);
        } catch (NumberFormatException nfe) {
            outputArea.setText("Error: Application ID must be a number.");
            return;
        }
        String status = (String) statusDropdown.getSelectedItem();
        String sql = "UPDATE applications SET status = ? WHERE app_id = ?";
        try (Connection con = connect();
             PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, status);
            ps.setInt(2, appId);
            int rows = ps.executeUpdate();
            if (rows == 0) {
                outputArea.setText("Error: No application found with app_id " + appId);
            } else {
                outputArea.setText("Application " + appId + " updated to status: " + status);
            }
            updateAppIdField.setText("");
        } catch (SQLException ex) {
            outputArea.setText("Error updating status: " + ex.getMessage());
        }
    }
```

```
    // ---------- Core Module 4: Search Applications by Job/Status ----------
    private JPanel buildSearchPanel() {
        JPanel panel = new JPanel(new GridLayout(3, 2, 5, 5));
        searchJobIdField = new JTextField();
        searchStatusDropdown = new JComboBox<>(new String[]{"Any", "Applied",
"Shortlisted", "Rejected", "Hired"});
        JButton searchBtn = new JButton("Search");

        panel.add(new JLabel("Job ID (blank = all jobs):"));
        panel.add(searchJobIdField);
        panel.add(new JLabel("Status:"));
        panel.add(searchStatusDropdown);
        panel.add(new JLabel(""));
        panel.add(searchBtn);

        searchBtn.addActionListener(e -> searchApplications());
        return panel;
    }

    // Join query: shows candidates against the job TITLE, not just job_id
    private void searchApplications() {
        String jobIdText = searchJobIdField.getText().trim();
        String status = (String) searchStatusDropdown.getSelectedItem();

        StringBuilder sql = new StringBuilder(
            "SELECT a.app_id, j.title, a.candidate_name, a.status " +
            "FROM applications a JOIN jobs j ON a.job_id = j.job_id WHERE 1=1");
        if (!jobIdText.isEmpty()) sql.append(" AND a.job_id = ?");
        if (!status.equals("Any")) sql.append(" AND a.status = ?");

        try (Connection con = connect();
```

```
         PreparedStatement ps = con.prepareStatement(sql.toString())) {
        int idx = 1;
        if (!jobIdText.isEmpty()) ps.setInt(idx++, Integer.parseInt(jobIdText));
        if (!status.equals("Any")) ps.setString(idx++, status);

        try (ResultSet rs = ps.executeQuery()) {
            StringBuilder sb = new StringBuilder("App ID | Job Title | Candidate | Status\n");
            boolean any = false;
            while (rs.next()) {
                any = true;
                sb.append(rs.getInt("app_id")).append(" | ")
                  .append(rs.getString("title")).append(" | ")
                  .append(rs.getString("candidate_name")).append(" | ")
                  .append(rs.getString("status")).append("\n");
            }
            outputArea.setText(any ? sb.toString() : "No matching applications found.");
        }
    } catch (SQLException | NumberFormatException ex) {
        outputArea.setText("Error searching: " + ex.getMessage());
    }
}

// ---------- Core Module 5: Shortlisted Candidates Report ----------
private JPanel buildReportPanel() {
    JPanel panel = new JPanel(new BorderLayout());
    JButton reportBtn = new JButton("Generate Shortlisted Report");
    panel.add(reportBtn, BorderLayout.NORTH);
    reportBtn.addActionListener(e -> shortlistedReport());
    return panel;
}

private void shortlistedReport() {
```

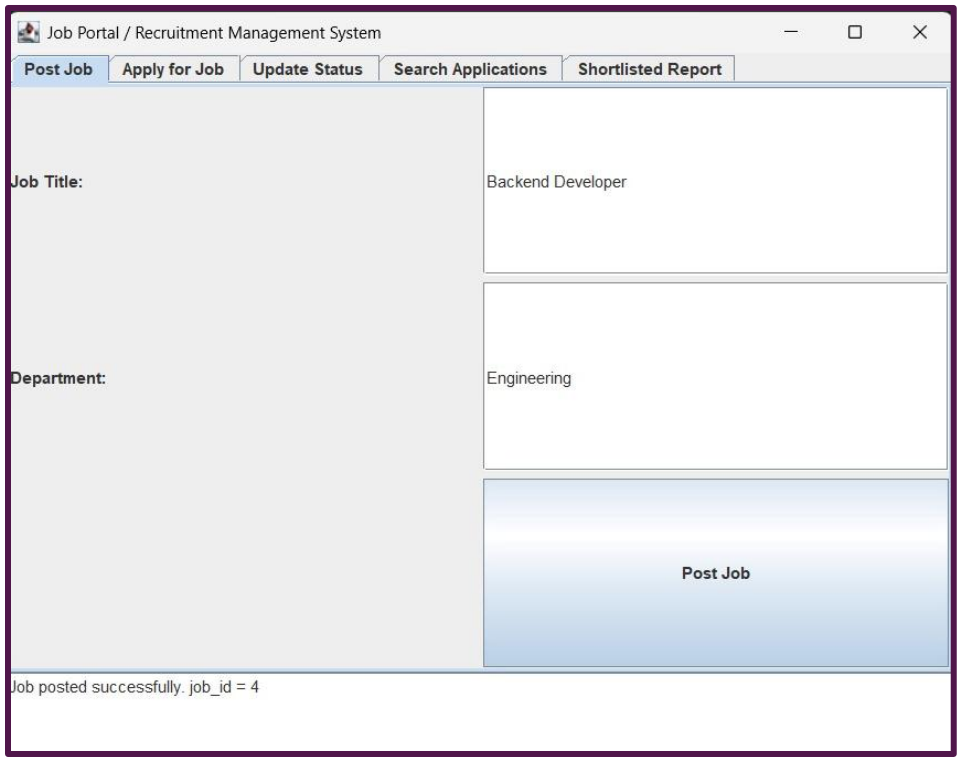
```
        String sql = "SELECT j.title, a.candidate_name " +
                     "FROM applications a JOIN jobs j ON a.job_id = j.job_id " +
                     "WHERE a.status = 'Shortlisted' ORDER BY j.title";
        try (Connection con = connect();
             PreparedStatement ps = con.prepareStatement(sql);
             ResultSet rs = ps.executeQuery()) {
            StringBuilder sb = new StringBuilder("Job Title | Shortlisted Candidate\n");
            boolean any = false;
            while (rs.next()) {
                any = true;
                sb.append(rs.getString("title")).append(" |
").append(rs.getString("candidate_name")).append("\n");
            }
            outputArea.setText(any ? sb.toString() : "No shortlisted candidates yet.");
        } catch (SQLException ex) {
            outputArea.setText("Error generating report: " + ex.getMessage());
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new JobPortalApp().setVisible(true));
    }
}
```

5. TEST CASES

#	Input	Expected Outcome	Actual Outcome
1 (typical)	Post job "Backend Developer" / "Engineering" (job_id auto-generated), then apply candidate "Arjun R" to that job_id	Job inserts; application inserts with status "Applied"	Job posted (job_id = 1). Application submitted for "Arjun R" — app_id = 1, status "Applied". Matches expected outcome.
2 (edge case)	Update that application's status to "Shortlisted", then Search by that job_id	Search returns one row: App ID, "Backend Developer", "Arjun R", "Shortlisted" — title comes from the join, not the raw job_id	Status updated to "Shortlisted". Search by job_id 1 returned: "1 | Backend Developer | Arjun R | Shortlisted" — title correctly pulled via JOIN. Matches expected.
3 (invalid/failure)	Apply a candidate to a non-existent job_id (e.g. 9999)	Application rejected with error "No job exists with job_id 9999"; no row inserted into applications	Application to job_id 9999 rejected with error "No job exists with job_id 9999"; no row inserted. Matches expected.

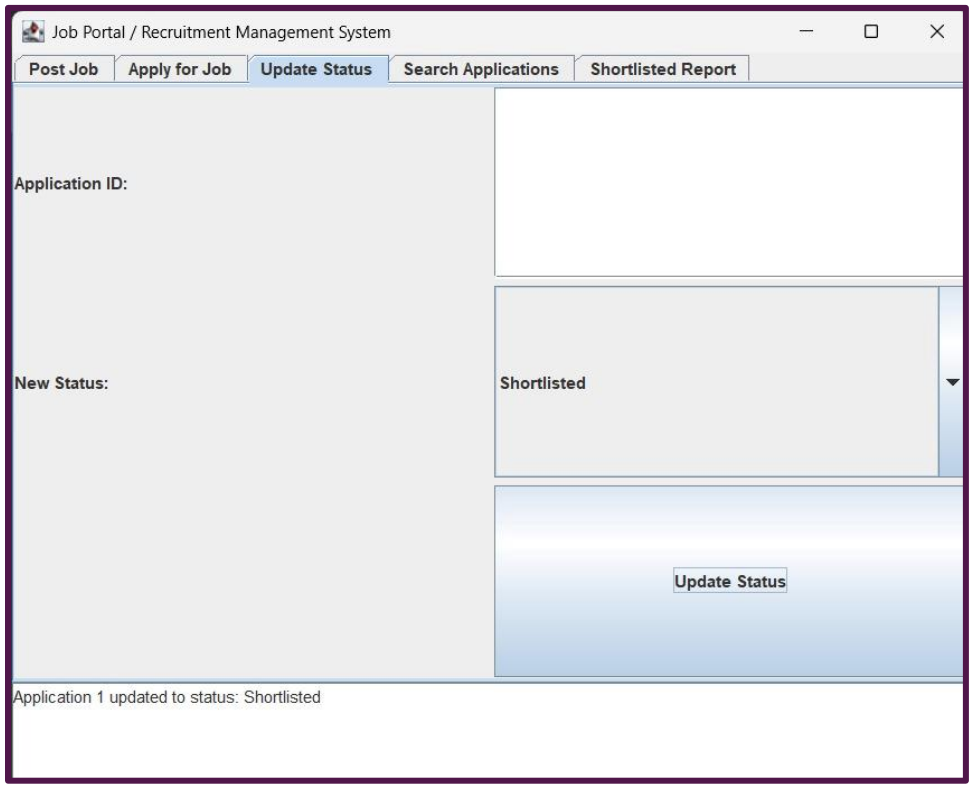
6. SCREENSHOTS OF EXECUTION

TEST CASE 1 :

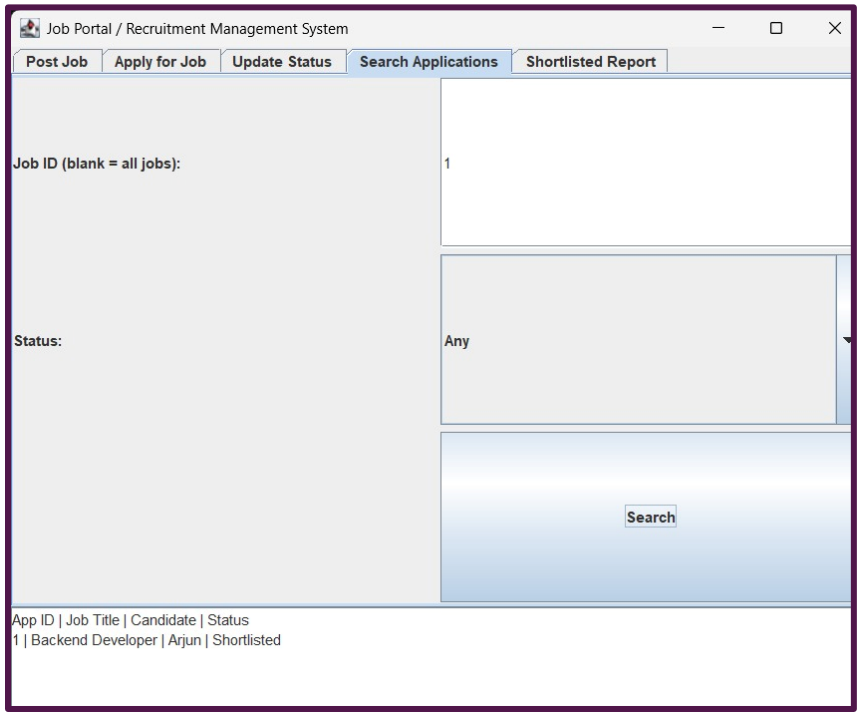


TEST CASE 2 :

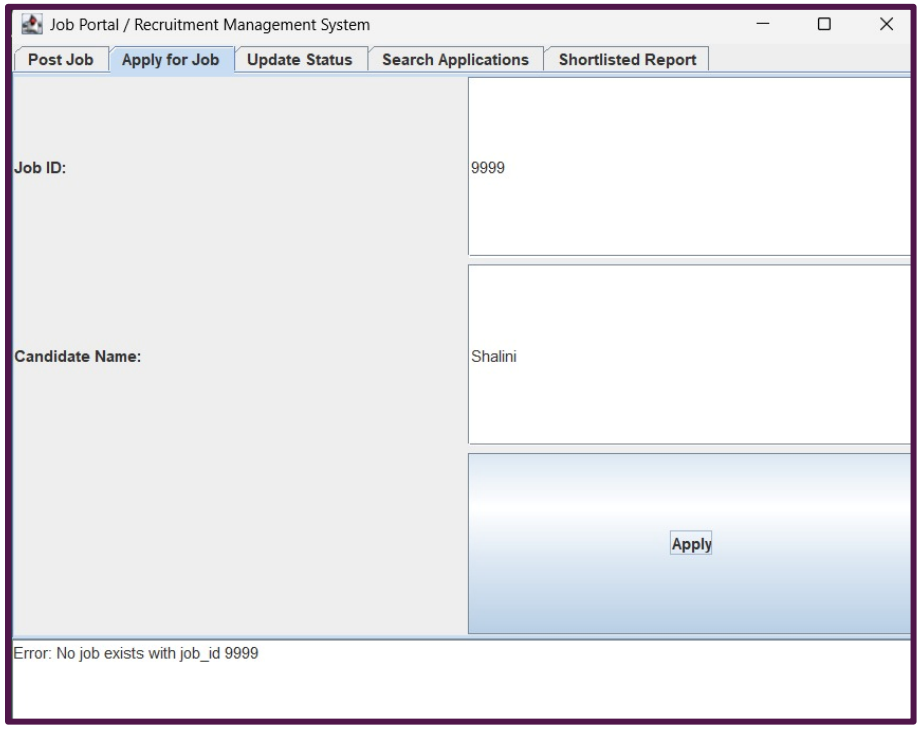
Part – 1:



Part – 2 :



TEST CASE – 3:



7. REFLECTION

When I applied for a non-existent job_id, I first validated it using a SELECT query before attempting the INSERT. This provided a clear and readable error message, such as **"No job exists with job_id 9999,"** instead of displaying a raw SQLException caused by a foreign key constraint violation. Testing all three cases also showed the importance of using a JOIN in the search query, as it displays readable job titles instead of only showing raw job_id values, making the results more useful and understandable in the GUI.

8. DECLARATION

I confirm that this submission, including the source code, design approach, and documentation, is my own work. AI assistance (Claude) was used to help structure the documentation and organize the test cases. However, the code logic was written, reviewed, and fully understood by me before submission.